VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Quiz Review: Pipelining

Advanced System Architectures

CO5059

Instructor: Tran Ngoc Thinh

Notes by: Phan Nhat Minh (minh.phannhat95@hcmut.edu.vn)

Semester: 261

Lecture 3 – pipeline timing, speedup, CPI with stalls, structural/data/control hazards, forwarding, branch schemes. A cheat sheet, the slide exercises with solutions, exam-style questions, and extra practice problems – with solutions at the end.

Last updated: October 4, 2026

Contents

1 Cheat Sheet **1**
- 1.1 Timing . . . 1
- 1.2 The 5-stage MIPS pipeline . . . 1
- 1.3 Data hazards: how many stalls? . . . 1
- 1.4 Drawing a pipeline diagram . . . 2
- 1.5 Control hazards . . . 2
- 1.6 Structural hazards . . . 2

2 Exercises from the Slides (with Solutions) **2**

3 Exam-Style Questions **5**

4 Practice Problems **6**

5 Solutions to the Practice Problems **8**

6 Solutions to the Exam-Style Questions **9**

1 Cheat Sheet

1.1 Timing

Theorem 1.1 (Cycles and time). For n instructions on a k-stage pipeline with s total stall cycles:

$$\text{Cycles} = k + (n-1) + s, \qquad \text{Time} = \text{Cycles} \times t_{clk},$$

$$t_{clk} = \max_i(\text{stage delay}_i) + \text{latch overhead}, \qquad \text{Latency of one instruction} = k \cdot t_{clk}.$$

Theorem 1.2 (Speedup). Ideal (balanced stages, p = unpipelined time per task):

$$\text{Speedup} = \frac{n\,p}{\frac{p}{k}(n-1)+p} = \frac{nk}{n+k-1} \xrightarrow{n\to\infty} k.$$

With stalls:

$$\text{CPI}_{pipe} = 1 + \text{stall cycles per instruction}$$

$$\text{Speedup} = \frac{\text{CPI}_{unpipe}}{\text{CPI}_{pipe}} \times \frac{t_{clk,unpipe}}{t_{clk,pipe}} = \frac{\text{Depth}}{1 + \text{stall CPI}} \quad \text{(balanced stages)}$$

Note. Pipelining improves **throughput**, not **latency**. Ideal CPI = 1 because one instruction *completes* per cycle once the pipeline is full: $\text{CPI} = (k+n-1)/n \to 1$.

1.2 The 5-stage MIPS pipeline

IF (instruction memory) – ID (decode, read registers) – EX (ALU / address) – MEM (data memory) – WB (write register). Registers are **written in the first half** and **read in the second half** of a cycle, so an instruction can read in ID a value written by WB in the same cycle.

1.3 Data hazards: how many stalls?

Let d be the distance from producer to consumer ($d = 1$: the very next instruction).

Situation	Stalls	Why
No forwarding, any producer	$\max(0,\, 3-d)$	consumer's ID must not precede producer's WB
Forwarding, ALU → ALU	0	EX/MEM → EX ($d=1$), MEM/WB → EX ($d=2$)
Forwarding, load → use ($d=1$)	1	data ready only at end of MEM
Forwarding, load → use ($d \geq 2$)	0	MEM/WB → EX
Forwarding, load → store data	0	MEM/WB → store input

Only **RAW** causes hazards in the in-order 5-stage pipeline; WAR and WAW cannot happen because reads (ID) are early and writes (WB) are late and in order.

1.4 Drawing a pipeline diagram

1. One row per instruction, one column per cycle; each row starts one cycle after the previous row's IF.
2. When an instruction stalls, write `stall` after its ID (it waits in ID); the instruction behind it waits in IF, and everything behind shifts right.
3. Total cycles = cycle of the last WB.

In wide diagrams below, `stl` abbreviates `stall`.

1.5 Control hazards

Proposition 1.3 (Branch cost).

$$\text{Branch penalty} = (\text{stage where branch is resolved}) - 1$$
$$\text{CPI} = 1 + f_{branch} \times (\text{fraction penalized}) \times \text{penalty}$$

Resolved in MEM ⇒ 3 cycles; moved to ID (zero test + extra adder) ⇒ 1 cycle.

Scheme	Cost
Stall (flush)	every branch pays the full penalty
Predict not taken	only *taken* branches pay (fetch PC+4, squash on taken); ≈ 47% of MIPS branches are not taken
Predict taken	untaken branches pay; in the 5-stage MIPS still 1 cycle because the target is not known earlier
Delayed branch	the instruction in the delay slot always executes; cost = wasted (unfilled/useless) slots

Delay-slot filling: (A) from before the branch – best, always useful, the branch must not depend on it; (B) from the target – useful when taken, may need copying, must be safe if not taken; (C) from the fall-through – useful when not taken, must be safe if taken. Compilers fill ≈ 60% of slots, ≈ 80% of those usefully ⇒≈ 50% useful.

1.6 Structural hazards

Two instructions need the same resource in the same cycle (e.g. one memory port for instructions and data). Fix: duplicate the resource (split I/D memory) or stall. Sometimes accepted when the hazard is rare and avoiding it is costly.

2 Exercises from the Slides (with Solutions)

Exercise 2.1 (Laundry). Wash 30 min, dry 40 min, fold 20 min; 4 loads. Sequential vs. pipelined time and speedup?

Solution. Sequential: $4 \times 90 = 360$ min $= 6$ h. Pipelined: the 40-min dryer sets the pace: $30 + 4 \times 40 + 20 = 210$ min $= 3.5$ h. Speedup $= 6/3.5 = 1.7$ (not 3), because stages are unbalanced and the pipeline must fill and drain. □

Exercise 2.2 (Single-cycle vs. pipelined). Stage times IF 2, Reg 1, ALU 2, Mem 2, WB 1 ns. Compare 3 loads, then 1003 loads, on a single-cycle (8 ns) and a pipelined (2 ns) machine.

Solution. 3 loads: single-cycle $3 \times 8 = 24$ ns; pipelined $(5 + 2) \times 2 = 14$ ns; speedup 1.7. 1003 loads: single-cycle $1003 \times 8 = 8024$ ns; pipelined $(5 + 1002) \times 2 = 2014$ ns; speedup $3.98 \approx 8/2 = 4$. The speedup approaches the ratio of clock periods (not 5, because the stages are unbalanced: the 1-ns stages still take a 2-ns cycle). □

Exercise 2.3 (Pipelining a multi-cycle CPU). Unpipelined: 1 ns clock, ALU/branch 4 cycles, memory 5 cycles; mix 40% ALU, 20% branch, 40% memory. Pipelining adds 0.2 ns to the clock. Speedup?

Solution. Unpipelined average $= 1 \times (0.6 \times 4 + 0.4 \times 5) = 4.4$ ns. Pipelined average $= 1.2$ ns (ideal CPI $= 1$). Speedup $= 4.4/1.2 = 3.7$. □

Exercise 2.4 (Structural hazard). Loads/stores are 40% of instructions; the machine with one memory port stalls 1 cycle per data access but has a 1.05× faster clock. Which is better?

Solution. Hazard machine: $(1 + 0.4 \times 1) \times t/1.05 = 1.33\,t$ versus $1.0\,t$ for the ideal machine, so the machine without the hazard is 1.33× faster. □

Exercise 2.5 (RAW hazard on R1). `add r1,r2,r3; sub r4,r1,r3; and r6,r1,r7; or r8,r1,r9; xor r10,r1,r11`. Stalls and total cycles without and with forwarding?

Solution. Without forwarding `sub` ($d = 1$) needs 2 stalls; after that every later reader is far enough behind:

```
               c1    c2    c3    c4    c5    c6    c7    c8    c9    c10   c11
add r1,r2,r3   IF    ID    EX    MEM   WB
sub r4,r1,r3         IF    ID    stall stall EX    MEM   WB
and r6,r1,r7               IF    stall stall ID    EX    MEM   WB
or  r8,r1,r9                           IF    ID    EX    MEM   WB
xor r10,r1,r11                               IF    ID    EX    MEM   WB
```

11 cycles. With forwarding (EX/MEM → EX to `sub`, MEM/WB → EX to `and`, register file for `or`/`xor`): 0 stalls, $5 + 4 = 9$ cycles. □

Exercise 2.6 (Load-use hazard). `lw r1,0(r2); sub r4,r1,r6; and r6,r1,r7; or r8,r1,r9` with forwarding.

Solution. The loaded value exists only at the end of MEM (cycle 4) but `sub` needs it at the start of its EX (cycle 4): 1 stall is unavoidable.

```
             c1    c2    c3    c4    c5    c6    c7    c8    c9
lw  r1,0(r2) IF    ID    EX    MEM   WB
sub r4,r1,r6       IF    ID    stall EX    MEM   WB
and r6,r1,r7             IF    stall ID    EX    MEM   WB
```

```
or  r8,r1,r9                    IF    ID    EX    MEM   WB
```

9 cycles ($5 + 3 + 1$). □

Exercise 2.7 (Compiler scheduling). `a=b+c; d=e-f;` compiled as `lw Rb; lw Rc; add Ra; sw Ra; lw Re; lw Rf; sub Rd; sw Rd` (forwarding, 1-cycle load delay). Stalls before and after scheduling?

Solution. Original: `add` right after `lw Rc` and `sub` right after `lw Rf` ⇒ 2 stalls, 5+7+2 = 14 cycles. Scheduled `lw Rb; lw Rc; lw Re; add Ra; lw Rf; sw Ra; sub Rd; sw Rd`: no load is immediately followed by its use ⇒ 0 stalls, 12 cycles. □

Exercise 2.8 (Branch stall impact). 30% of instructions are branches with a 3-cycle stall. New CPI?

Solution. CPI $= 1 + 0.30 \times 3 = 1.9$ – almost twice the ideal. With the branch resolved in ID (1 cycle): $1 + 0.3 = 1.3$. □

Exercise 2.9 (Comparing branch schemes). 4% unconditional, 6% conditional untaken, 10% conditional taken; pipeline depth 5. Find CPI and speedups for stall, predict not taken, predict taken, delayed branch (penalty 0.5).

Solution.

Scheme	CPI	vs. unpipelined	vs. stall
Stall: $1 + 0.20 \times 3$	1.60	$5/1.6 = 3.1$	1.00
Not taken: $1 + 0.04 \times 1 + 0.10 \times 3$	1.34	3.7	1.19
Taken: $1 + 0.14 \times 1 + 0.06 \times 2$	1.26	4.0	1.27
Delayed: $1 + 0.20 \times 0.5$	1.10	4.5	1.45

(The slide lists 1.29 for predict taken, which is the ratio of the rounded speedups 4.0/3.1; the exact value is $1.6/1.26 = 1.27$.) □

Exercise 2.10 (CPI of a realistic mix). Loads 30% (25% followed immediately by a use), branches 20% (45% taken); forwarding, branch resolved in ID, predict not taken.

Solution. CPI $= 1 + 0.3 \times 0.25 \times 1 + 0.2 \times 0.45 \times 1 = 1.165$. □

Exercise 2.11 (Hazards, NOPs and forwarding).

```
(1) lw  $1, 40($6)
(2) add $6, $2, $2
(3) sw  $6, 50($1)
(4) add $4, $5, $6
(5) lw  $6, 10($4)
```

(A) Find all data dependencies. (B) Without forwarding, which cause hazards? (C) Insert NOPs so the code runs correctly without forwarding. (D) How many cycles does (C) take? (E) With full forwarding, how many cycles, and which forwarding paths are used?

Solution. **(A)** RAW: (1)→(3) on `$1`; (2)→(3) on `$6`; (2)→(4) on `$6`; (4)→(5) on `$4`. WAR: (1)→(2), (3)→(5), (4)→(5) on `$6`. WAW: (2)→(5) on `$6`.

(B) Only RAW with $d \le 2$: (1)→(3) ($d = 2$), (2)→(3) ($d = 1$), (2)→(4) ($d = 2$), (4)→(5) ($d = 1$). WAR/WAW are not hazards here.

(C) Each consumer must be at least 3 slots after its producer:

```
lw  $1, 40($6)
add $6, $2, $2
nop
nop
sw  $6, 50($1)
add $4, $5, $6
nop
nop
lw  $6, 10($4)
```

(D) 9 instructions ⇒ $5 + 8 = 13$ cycles:

```
               c1   c2   c3   c4   c5   c6   c7   c8   c9   c10  c11  c12  c13
lw  $1,40($6)  IF   ID   EX   MEM  WB
add $6,$2,$2        IF   ID   EX   MEM  WB
nop                 IF   ID   EX   MEM  WB
nop                      IF   ID   EX   MEM  WB
sw  $6,50($1)            IF   ID   EX   MEM  WB
add $4,$5,$6                  IF   ID   EX   MEM  WB
nop                           IF   ID   EX   MEM  WB
nop                                IF   ID   EX   MEM  WB
lw  $6,10($4)                      IF   ID   EX   MEM  WB
```

(E) No stalls are needed (the only load, (1), is used two instructions later), so $5 + 4 = 9$ cycles:

```
               c1    c2    c3    c4    c5    c6    c7    c8    c9
lw  $1,40($6)  IF    ID    EX    MEM   WB
add $6,$2,$2         IF    ID    EX    MEM   WB
sw  $6,50($1)              IF    ID    EX    MEM   WB
add $4,$5,$6               IF    ID    EX    MEM   WB
lw  $6,10($4)                    IF    ID    EX    MEM   WB
```

Forwarding paths: cycle 5, MEM/WB → EX gives `$1` (from (1)) and EX/MEM → EX gives `$6` (from (2)) to the `sw`; cycle 6, MEM/WB → EX gives `$6` (from (2)) to (4); cycle 7, EX/MEM → EX gives `$4` (from (4)) to (5). □

3 Exam-Style Questions

Multi-part questions in the style of the in-class exercise. Solutions are in the last section.

Exercise 3.1 (Pipeline timing). A 5-stage pipeline has stage latencies IF 250, ID 200, EX 300, MEM 350, WB 150 ps. Each pipeline register adds 20 ps.

(a) What is the clock period of the non-pipelined (single-cycle) version?

(b) What is the clock period of the pipelined version?

(c) What is the latency of one instruction in the pipeline?

(d) How long do 10^6 instructions take on each version (no stalls)? What is the speedup?

(e) What is the pipelined throughput in instructions per second?

(f) MEM is split into two stages of 175 ps each. Find the new clock period and the speedup over the 5-stage pipeline for 10^6 instructions.

Exercise 3.2 (Hazards and forwarding).

```
(1) lw  $s2, 0($s1)
(2) add $s3, $s2, $s4
(3) sub $s5, $s3, $s2
(4) sw  $s5, 4($s1)
(5) add $t0, $s6, $s7
```

(a) List all RAW dependencies, with their distance.

(b) Which of them cause hazards without forwarding?

(c) Insert NOPs so the code runs correctly without forwarding. How many cycles does it take?

(d) With full forwarding, draw the pipeline diagram. How many stalls and cycles?

(e) Name the forwarding path used for every dependency in (d).

(f) Reorder the code (same result) so it runs with no stalls when forwarding is available. How many cycles now?

Exercise 3.3 (Branches and CPI). A program's mix: 25% loads, 10% stores, 15% branches (60% taken), 50% ALU. 40% of loads are immediately followed by an instruction that uses the loaded value. The pipeline has full forwarding.

(a) Branches are resolved in MEM and the pipeline stalls on every branch. CPI?

(b) Predict not taken, branch resolved in MEM. CPI?

(c) Predict not taken, branch resolved in ID. CPI?

(d) One delay slot (branch resolved in ID); the compiler fills 70% of slots with useful instructions. CPI?

(e) With a 1 GHz clock and 10^9 instructions, find the execution time for (a) and (c) and the speedup.

(f) An unpipelined multi-cycle version runs at the same clock with CPI = 4.6. What speedup does design (c) give over it?

4 Practice Problems

Exercise 4.1. Stage delays: IF 200, ID 150, EX 250, MEM 300, WB 150 ps; latch overhead 20 ps. (a) Single-cycle clock? (b) Pipelined clock? (c) Latency of one instruction when

pipelined? (d) Time for 1000 instructions on each, and the speedup? (e) MEM is split into two 150-ps stages. New clock and speedup over (d) for 1000 instructions?

Exercise 4.2. For a 5-stage pipeline, compute the ideal speedup for $n = 10$, $n = 100$ and $n \to \infty$ instructions.

Exercise 4.3.

```
(1) add $t0, $t1, $t2
(2) lw  $t3, 0($t0)
(3) sub $t4, $t3, $t5
(4) sw  $t4, 4($t0)
(5) or  $t6, $t0, $t4
```

List the RAW dependencies. Draw the diagram and count cycles (a) without forwarding, (b) with full forwarding.

Exercise 4.4. With forwarding, how many cycles does this take? Reorder it (without changing the result) to remove all stalls.

```
lw  $t0, 0($a0)
add $t1, $t0, $t2
lw  $t3, 4($a0)
add $t4, $t3, $t2
sw  $t1, 0($a1)
```

Exercise 4.5. 25% of instructions are branches and 60% of them are taken. Find the CPI for: (a) stall, branch resolved in MEM; (b) predict not taken, resolved in MEM; (c) predict not taken, resolved in ID; (d) one delay slot that is usefully filled 50% of the time.

Exercise 4.6. Loads 30% (40% of them followed immediately by a use), branches 15% (50% taken, 1-cycle penalty on taken), full forwarding. Find the CPI. An unpipelined version has CPI 4.5 with a 1 ns clock; the pipelined clock is 1.1 ns. Speedup?

Exercise 4.7. 35% of instructions access data memory. With a single memory port each access causes a 1-cycle stall. A dual-port design avoids this but its clock period is 10% longer. Which is faster and by how much?

Exercise 4.8. Classify every dependency (RAW/WAR/WAW). Which ones are hazards in the 5-stage MIPS pipeline?

```
(1) add $1, $2, $3
(2) sub $4, $1, $5
(3) and $1, $6, $7
(4) or  $8, $1, $4
```

Exercise 4.9. Can `add $s1,$s2,$s3` be moved into the delay slot of the following `beq $t0,$t1,L`? What if it were `add $t0,$s2,$s3` instead?

Exercise 4.10. Draw the diagram with full forwarding, mark the stall(s), and name the forwarding path for every dependency:

```
(1) lw  $2, 20($1)
(2) and $4, $2, $5
(3) or  $8, $2, $6
(4) add $9, $4, $2
(5) slt $1, $6, $7
```

5 Solutions to the Practice Problems

Solution (4.1). (a) $200 + 150 + 250 + 300 + 150 = 1050$ ps. (b) $300 + 20 = 320$ ps (slowest stage plus latch). (c) $5 \times 320 = 1600$ ps – longer than single-cycle; pipelining does not cut latency. (d) Single-cycle $1000 \times 1050 = 1{,}050{,}000$ ps; pipelined $(5+999) \times 320 = 321{,}280$ ps; speedup 3.27. (e) Slowest stage is now EX: $250 + 20 = 270$ ps; $(6+999) \times 270 = 271{,}350$ ps; speedup over (d) $= 321{,}280/271{,}350 = 1.18$. □

Solution (4.2). $nk/(n+k-1)$: $n = 10$: $50/14 = 3.57$; $n = 100$: $500/104 = 4.81$; $n \to \infty$: 5. □

Solution (4.3). RAW: (1)→(2), (1)→(4), (1)→(5) on `$t0`; (2)→(3) on `$t3`; (3)→(4), (3)→(5) on `$t4`.

(a) Without forwarding (each consumer ≥ 3 slots after its producer):

```
                c1   c2   c3   c4   c5   c6   c7   c8   c9   c10  c11  c12  c13  c14  c15
add $t0,$t1,$t2 IF   ID   EX   MEM  WB
lw  $t3,0($t0)       IF   ID   stl  stl  EX   MEM  WB
sub $t4,$t3,$t5           IF   stl  stl  ID   stl  stl  EX   MEM  WB
sw  $t4,4($t0)                      IF   stl  stl  ID   stl  stl  EX   MEM  WB
or  $t6,$t0,$t4                               IF   stl  stl  ID   EX   MEM  WB
```

6 stalls, 15 cycles.

(b) With forwarding only the load-use (2)→(3) stalls:

```
                 c1    c2    c3    c4    c5    c6    c7    c8    c9    c10
add $t0,$t1,$t2  IF    ID    EX    MEM   WB
lw  $t3,0($t0)         IF    ID    EX    MEM   WB
sub $t4,$t3,$t5              IF    ID    stall EX    MEM   WB
sw  $t4,4($t0)               IF    stall ID    EX    MEM   WB
or  $t6,$t0,$t4                    IF    ID    EX    MEM   WB
```

1 stall, 10 cycles. □

Solution (4.4). Two load-use stalls: $5 + 4 + 2 = 11$ cycles. Reordered:

```
lw  $t0, 0($a0)
lw  $t3, 4($a0)
```

```
add $t1, $t0, $t2
add $t4, $t3, $t2
sw  $t1, 0($a1)
```

Every use is now 2 instructions after its load: 0 stalls, 9 cycles. □

Solution (4.5). (a) $1+0.25\times3 = 1.75$. (b) $1+0.25\times0.6\times3 = 1.45$. (c) $1+0.25\times0.6\times1 = 1.15$. (d) $1 + 0.25 \times 0.5 = 1.125$. □

Solution (4.6). CPI $= 1+0.3\times0.4\times1+0.15\times0.5\times1 = 1+0.12+0.075 = 1.195$. Speedup $= (4.5\times1)/(1.195\times1.1) = 4.5/1.3145 = 3.42$. □

Solution (4.7). Single port: $1.35 \times t$. Dual port: $1.0 \times 1.1t = 1.1t$. Dual port is faster by $1.35/1.1 = 1.23\times$. □

Solution (4.8). RAW: (1)→(2) `$1`; (2)→(4) `$4`; (3)→(4) `$1` (instruction 4 reads the *new* `$1` from (3), not from (1)). WAR: (2)→(3) on `$1`. WAW: (1)→(3) on `$1`. Only the RAW dependencies can be hazards in the in-order 5-stage pipeline; with forwarding none of them stalls. □

Solution (4.9). Yes: the branch does not read `$s1`, so it is an independent instruction from before the branch (strategy A, always useful). No for `add $t0,…`: the branch depends on `$t0`, so the slot must be filled from the target (B) or fall-through (C) path instead. □

Solution (4.10).

```
                    c1    c2    c3    c4    c5    c6    c7    c8    c9    c10
lw  $2,20($1)  IF   ID    EX    MEM   WB
and $4,$2,$5        IF    ID    stall EX    MEM   WB
or  $8,$2,$6              IF    stall ID    EX    MEM   WB
add $9,$4,$2                    IF    ID    EX    MEM   WB
slt $1,$6,$7                          IF    ID    EX    MEM   WB
```

(1)→(2) `$2`: load-use, 1 stall, then MEM/WB → EX in cycle 5. (1)→(3) `$2`: register file (written in the first half of cycle 5, read in the second half). (2)→(4) `$4`: MEM/WB → EX in cycle 7. (1)→(4) `$2`: register file. (5) has no dependency. Total 10 cycles. □

6 Solutions to the Exam-Style Questions

Solution (3.1).

(a) $250 + 200 + 300 + 350 + 150 = 1250$ ps.

(b) Slowest stage + overhead $= 350 + 20 = 370$ ps.

(c) $5 \times 370 = 1850$ ps (longer than 1250 ps: latency gets worse).

(d) Non-pipelined: $10^6 \times 1250$ ps $= 1.25$ ms. Pipelined: $(5 + 10^6 - 1) \times 370$ ps ≈ 0.370 ms. Speedup $\approx 1250/370 = 3.38$ (well below 5 because the stages are unbalanced).

(e) One instruction per cycle: $1/370$ ps $\approx 2.7 \times 10^9$ instructions/s.

(f) The slowest stage is now EX: $300 + 20 = 320$ ps. Speedup $= \dfrac{(10^6+4)\times 370}{(10^6+5)\times 320} \approx 1.156$.

□

Solution (3.2).

(a) (1)→(2) `$s2` ($d=1$); (1)→(3) `$s2` ($d=2$); (2)→(3) `$s3` ($d=1$); (3)→(4) `$s5` ($d=1$). Instruction (5) is independent.

(b) All four have $d \le 2$, so all are hazards without forwarding.

(c) Two NOPs after each of (1), (2), (3): `lw; nop; nop; add; nop; nop; sub; nop; nop; sw; add` – 11 instructions, $5 + 10 = 15$ cycles. (With an interlock instead of NOPs the timing is the same:)

```
                c1   c2   c3   c4   c5   c6   c7   c8   c9   c10  c11  c12  c13  c14  c15
lw  $s2,0($s1)  IF   ID   EX   MEM  WB
add $s3,$s2,$s4      IF   ID   stl  stl  EX   MEM  WB
sub $s5,$s3,$s2           IF   stl  stl  ID   stl  stl  EX   MEM  WB
sw  $s5,4($s1)                      IF   stl  stl  ID   stl  stl  EX   MEM  WB
add $t0,$s6,$s7                          IF   stl  stl  ID   EX   MEM  WB
```

(d) Only the load-use (1)→(2) stalls: 1 stall, 10 cycles.

```
                c1   c2   c3   c4    c5   c6   c7   c8   c9   c10
lw  $s2,0($s1)  IF   ID   EX   MEM   WB
add $s3,$s2,$s4      IF   ID   stall EX   MEM  WB
sub $s5,$s3,$s2           IF   stall ID   EX   MEM  WB
sw  $s5,4($s1)                IF    ID   EX   MEM  WB
add $t0,$s6,$s7               IF    ID   EX   MEM  WB
```

(e) (1)→(2): MEM/WB → EX in cycle 5 (after the stall). (1)→(3): register file (written in the first half of cycle 5, read in the second half). (2)→(3): EX/MEM → EX in cycle 6. (3)→(4): EX/MEM → EX in cycle 7 (store data).

(f) Move the independent (5) between the load and its use: `lw; add $t0; add $s3; sub; sw`. The use of `$s2` is now 2 instructions after the load, so there are no stalls: 9 cycles.

```
                c1   c2   c3   c4   c5   c6   c7   c8   c9
lw  $s2,0($s1)  IF   ID   EX   MEM  WB
add $t0,$s6,$s7      IF   ID   EX   MEM  WB
add $s3,$s2,$s4           IF   ID   EX   MEM  WB
sub $s5,$s3,$s2                IF   ID   EX   MEM  WB
sw  $s5,4($s1)                      IF   ID   EX   MEM  WB
```

□

Solution (3.3). Load-use stalls per instruction $= 0.25 \times 0.4 \times 1 = 0.10$.

(a) $1 + 0.10 + 0.15 \times 3 = 1.55$.

(b) $1 + 0.10 + 0.15 \times 0.6 \times 3 = 1.37$.

(c) $1 + 0.10 + 0.15 \times 0.6 \times 1 = 1.19$.

(d) Wasted slots: $0.15 \times 0.3 \times 1 = 0.045$, so CPI $= 1.145$.

(e) $T_a = 10^9 \times 1.55 \times 1$ ns $= 1.55$ s; $T_c = 1.19$ s; speedup $= 1.55/1.19 = 1.30$.

(f) $4.6/1.19 = 3.87$.

□